

Julia 3D Inversion Development Plan

A phased software and scientific-validation roadmap for spatially coupled HE3D/MHS inversion, Wittmann EOS, FFNO non-LTE populations, Kurucz LTE synthesis, and extensible Julia spectral synthesis.

PURPOSE

This document is the shared execution contract for the human project owner and Codex. It defines what will be built, in which order, which interfaces must remain stable, and what evidence is required before each phase is accepted.

Core decision

The inversion is **not 1.5D**. Atmospheric node maps are optimized jointly over a 2D field of view. The only production architecture is hybrid MPI plus Julia threading: MPI distributes spatial tiles, threads process local columns, and global rank 0 alone launches and controls GPU work. If no magnetic field is available, the HE3D layer uses no B term. If B is available, the force balance includes the Lorentz term and the layer operates in magnetohydrostatic (MHS) mode. FFNO then predicts populations from the complete 3D atmospheric context and synthesis produces an image-spectral cube.

Document status	Target implementation	Prepared
Planning baseline v1.4	MPI/threaded Julia; GPU launched only by rank 0	25 Aug 2026

1. Outcome and scope

The deliverable is a Julia package and command-line application that estimates a 3D atmosphere from observed spectral image cubes. It owns node parameterization, HE3D/MHS reconstruction, EOS calls, population inference, line-physics selection, synthesis, objective evaluation, regularization, optimization, checkpointing, and diagnostics.

End-to-end forward model

```
node maps -> A(log tau500,x,y) -> HE3D/MHS + EOS + opacity -> A(z,x,y) -> {FFN03D non-LTE
populations + selected transitions | LTE populations + Kurucz line list} -> formal solution ->
instrument -> SpectralCube(lambda,Stokes,x,y); Release 1 activates I only and no PRD
```

In scope for the first production release

- Joint inversion of temperature and required velocity-component node maps; magnetic-field node maps are included only when B observations or a supplied B volume are available.
- An iterative force-balance reconstruction that updates P and rho and rebuilds tau500: HE3D without B, or MHS with the Lorentz force when B exists.
- Wittmann EOS integration for electron density and thermodynamic closure, plus a consistent 500 nm continuum-opacity implementation.
- Full-volume or halo-tiled FFNO inference. A pixel is never evaluated as if its horizontal neighborhood were irrelevant.
- Intensity-only synthesis and inversion first. The public spectral-cube contract still includes a Stokes axis so future I,Q,U,V support adds implementations rather than replacing interfaces.
- No partial redistribution (PRD) in the first release. Redistribution is selected through a stable line-physics interface whose initial implementation is non-PRD.
- A configurable LTE synthesis mode reads Kurucz line lists and synthesizes all selected/background lines in each spectral region without calling FFNOML. Spectral regions may select the FFNOML non-LTE backend or the Kurucz LTE backend independently, allowing simultaneous multi-region inversions.
- Coarse-to-fine spatial optimization, checkpoint/restart, synthetic validation, and uncertainty-oriented diagnostics.

Explicitly deferred

- PRD calculations, polarized Stokes synthesis/inversion, time-dependent inversion, and learned inverse initialization. PRD and Stokes are deferred implementations, not deferred architectural requirements.
- A fully differentiable Wittmann EOS on day one. Phase 1 favors correctness and numerical validation; gradient engineering follows measured bottlenecks.
- Replacing the existing trained FFNO model. Retraining on log tau500 or hydrostatically reconstructed inputs is a contingency triggered by validation evidence.

Scientific invariants

Invariant	Meaning / enforcement
Spatial coupling	FFNO receives the full domain or tiles with validated halos and overlap blending.
Coordinate consistency	The full $z(\log \tau_{500}, x, y)$ cube is retained; cycles return T, and optional B, to the same tau grid.
Thermodynamic consistency	Every T, optional-B or boundary-state update iterates Pgas and rho to self-consistency, then recomputes ne, z and populations.
Unit consistency	All public arrays carry declared units; conversion happens only at typed adapter boundaries.
Physics extensibility	FFNOML non-LTE and Kurucz LTE are selectable population/line-opacity backends. Intensity/non-PRD are capabilities, not baked-in shapes; PRD and IQUV use stable interfaces.
Reproducibility	Configuration, checkpoint hash, code revision, random seed and objective history are written with every run.

2. Architecture

Recommended package name: **FFNOInversion.jl**. Keep the inversion layer separate from research scripts and make the existing **Forward.jl** synthesis kernel a callable dependency rather than a file-driven program.

Module map

Julia module	Responsibility	Must not own
Types	Grid, units, atmosphere, observation, node layout and result types.	Physics or file-format logic.
Nodes	Bounded node maps; vertical interpolation; coarse/fine horizontal control grids.	EOS or synthesis.
ForceBalance	HE3D without B; MHS with Lorentz force when B exists; fixed-point Pgas/rho/tau/z loop and remapping.	FFNO or optimizer.
EOS	Stable Wittmann wrapper and batched thermodynamic queries.	Hidden global state.
Opacity500	Continuum extinction at 500 nm, with units and reference tests.	Line synthesis.
PopulationModels	Abstract population interface; FFNO non-LTE, LTE EOS, mock and recorded backends.	HDF5 in the hot loop.
LinePhysics	Transition selection, Kurucz line-list adapter and abstract redistribution policy; non-PRD now, PRD later.	Optimizer or data I/O.
Synthesis	Abstract scalar/polarized solver; FFNO-population and Kurucz-LTE intensity paths; atomic/radiation caches.	CLI parsing or a fixed Stokes count.
Observations	PSF, Mueller hook, masks, per-Stokes weights, resampling and normalization.	Atmospheric optimization.
Objectives	Component-aware spectral residuals, priors and 3D regularization.	Assuming intensity is the only observable.
Solvers	Multiscale optimizer, line search/trust region, stopping and recovery.	Physics implementation.
IO	Configuration, HDF5/Zarr-like run state, checkpoints and provenance.	Numerical decisions.
Diagnostics	Convergence plots, residual cubes, response/gradient checks and reports.	Forward-model mutation.

Dependency direction

CLI -> Solvers -> Objectives -> ForwardModel -> {Nodes, ForceBalance, PopulationModels, LinePhysics, Synthesis, Observations}. Low-level physics modules depend only on Types and explicit configuration. This direction must remain acyclic.

Initial language bridge

Global MPI rank 0 owns the persistent PyTorch inference service and is the only rank permitted to launch GPU processes. Rank 0 stages the distributed atmospheric input and returns population tiles to their owning ranks without checkpoint reload or temporary HDF5 in the hot loop. Use PythonCall.jl first unless deployment constraints favor a small long-lived RPC service. Record/replay and mock backends make CPU tests independent of a GPU.

LTE / Kurucz synthesis mode

Each spectral region declares **population_mode = ffnoml_nlte | lte**. The LTE mode bypasses FFNOML, computes LTE ionization/excitation populations from the reconstructed T, Pgas and ne, reads one or more configured Kurucz line-list files, selects lines overlapping the synthesis window plus a declared wing margin, and evaluates line opacity/emissivity with an LTE source function. Line-list parsing, wavelength convention, species/ion identifiers, excitation energies, log(gf), damping fields, abundance set, broadening policy and file checksum are explicit provenance. The same formal-solver and observation interfaces are used by both modes, so LTE does not become a second forward-model architecture.

Future-proof spectral-physics seams

Concern	Release 1	Stable extension contract
Redistribution	Non-PRD only.	AbstractRedistributionModel per transition; PRD may own angle/frequency quadrature, radiation state, iteration controls and caches without changing ForwardModel.
Population / line source	Per region: FFNOML non-LTE selected transitions or LTE Kurucz line list.	AbstractPopulationModel and AbstractLineOpacityModel select the backend; LTE bypasses FFNOML but rejoins the shared formal solver.
Polarization	Fit Stokes I only.	SpectralCube always has a component axis; StokesSet declares active components. A polarized solver returns I,Q,U,V through the same synthesise! contract.
Instrument	Scalar PSF.	Observation operator accepts component-aware kernels and optional Mueller response; current identity polarization response is explicit.
Objective	Only I is active.	Residuals/weights are indexed by region, Stokes component, wavelength and pixel; activating Q/U/V is configuration-driven.
Atomic model	Current metadata.	Transition metadata declares redistribution and polarization capabilities; unsupported requests fail validation.
Checkpoint	Intensity/non-PRD manifest.	Versioned capability manifest prevents silent restart under different line physics.

3. Adopted HE3D / MHS algorithm

The force-balance layer follows the iterative approach supplied by the project owner. The inversion state always provides **T**, **rho0** and **P0**; **B** is **optional**. Pressure and density are alternated until mutually consistent. If B is absent, it is not used or synthesized. If B is present, the Lorentz force is added and the solve is magnetohydrostatic (MHS). T and any available B remain attached to the same optical-depth points throughout the iteration.

Mode selection

Available input	Mode	Force-balance equation used by the plan
T, rho0, P0; no B	HE3D	$\text{grad}(P) = -\rho \cdot g$, using the sign convention of the supplied approach.
T, B, rho0, P0	MHS	$\text{grad}(P) = -\rho \cdot g + J \times B$, with $J = \text{curl}(B)/\mu_0$; signs must follow the same declared gravity convention.

Fixed-point cycle

Step	Input	Operation	Output
0. Initialize	T(tau), optional B(tau), rho0, P0	Select HE3D or MHS, validate boundaries, and construct the initial 3D tau/z mapping.	T, optional B, rho_i, P_i
1. Pressure solve	T, rho_i, optional B, P boundary	HE3D: solve $\text{grad}(P) = -\rho \cdot g$. MHS: solve $\text{grad}(P) = -\rho \cdot g + J \times B$.	P_{i+1}
2. EOS density	T, P_{i+1}	Use Wittmann EOS to calculate rho_{i+1}, ne and supporting thermodynamic quantities.	rho_{i+1}, ne_{i+1}
3. Optical depth	T, optional B, rho_{i+1}, P_{i+1}	Recalculate 500 nm opacity and integrate the new tau_{i+1} and z mapping.	tau_{i+1}, z_{i+1}
4. Preserve inversion variables	T(tau_i), optional B(tau_i), tau_{i+1}	Remap T and, only when present, B so values remain fixed at target tau points.	T(tau_target), optional B(tau_target)
5. Convergence	P_i, rho_i, P_{i+1}, rho_{i+1}	Evaluate scaled pressure/density residuals and the hydrostatic equation residual.	Converged atmosphere or next iteration

Iteration definition

```

Given target grid tau*, fixed T*(tau*) and optional B*(tau*): mode = isnothing(B*) ? HE3D : MHS
rho_i, P_i, mapping_i = initialize(T*, B*, rho0, P0) repeat force_i = mode == HE3D ? -rho_i*g :
-rho_i*g + curl(B*) x B* / mu0 P_{i+1} = solve_pressure_3d(force_i, boundary_conditions)
rho_{i+1}, ne_{i+1} = wittmann_eos(T*, P_{i+1}) tau_{i+1}, z_{i+1} = rebuild_tau500(T*, P_{i+1},
rho_{i+1}) T*, B* = preserve_on_target_tau(T*, optional B*, tau_{i+1}, tau*) check convergence
of P, rho and the selected HE3D/MHS force residual until converged return T*, optional B*, P,
rho, ne, tau*, z

```

Interpretation and boundary conditions

The photographed equation is **$\text{grad}(P) = -\rho \cdot g$** . With gravity purely vertical, its horizontal components imply zero horizontal gas-pressure gradient. In MHS mode the Lorentz force can support horizontal pressure structure, but the supplied B must still make the scalar pressure equation integrable. The implementation must test compatibility and boundary conditions; it must not silently change equations or manufacture B.

- Boundary inputs rho0 and P0 must declare their location: scalar, top surface, bottom surface, or full boundary field.
- Pressure positivity is enforced by the numerical solve; interpolation must be monotonic and conservative enough to avoid tau-grid oscillations.
- Convergence requires max relative changes in both P and rho below tolerances, a small normalized hydrostatic residual, and a stable tau/z mapping.
- T is never relaxed by force balance. In MHS mode B is also fixed during the inner iteration; only the outer inversion optimizer changes these fields.

- When B is unavailable, the magnetic arrays, Lorentz calculation and B remapping are all skipped. The code must not substitute zero-valued B merely to satisfy an interface.

HE3D diagnostics retained for every forward evaluation

Diagnostic	Purpose
Iteration count and convergence flag	Detect difficult columns/volumes and rejected optimizer trials.
$\ P_{i+1}-P_i\ $ and $\ \rho_{i+1}-\rho_i\ $	Verify fixed-point contraction rather than apparent EOS-only convergence.
Force residual	HE3D: $\ \text{grad}(P)+\rho*g\ $; MHS: $\ \text{grad}(P)+\rho*g-JxB\ $.
tau and z displacement per cycle	Reveal remapping instability and large coordinate corrections.
T / optional-B remap error	Guarantee T preservation and B preservation only in MHS mode.

4. Core contracts and data conventions

Stable contracts prevent later phases from becoming repository-wide rewrites. Depth is the first spatial dimension inside Julia unless the adopted synthesis library requires otherwise; all adapters must state permutations explicitly.

Object	Canonical shape	Type / unit	Notes
log_tau500	(nz,)	Float64, dimensionless	Monotonic; top to bottom convention documented and asserted.
T	(nz,nx,ny)	Float64, K	Optimized through node maps; bounded.
vx, vy, vz, vturb	(nz,nx,ny)	Float64, m s ⁻¹	Convert once at adapters if training used another unit.
Bx, By, Bz	optional (nz,nx,ny)	Float64, tesla	Absent in HE3D; present, preserved and used for Lorentz force in MHS.
Pgas	(nz,nx,ny)	Float64, Pa	Derived, except top-boundary parameter.
rho	(nz,nx,ny)	Float64, kg m ⁻³	Derived by EOS and selected HE3D/MHS mode.
ne	(nz,nx,ny)	Float64, m ⁻³	Positive; derived by Wittmann.
z	(nz,nx,ny)	Float64, m	Column-dependent/corrugated; declared zero convention.
populations	(nz,nx,ny,nlevels)	Float32/64, m ⁻³	Level map and atom metadata are mandatory.
spectral cube	(nlam,nstokes,nx,ny)	Float64 or normalized	nstokes=1, StokesSet=[I] now; future [I,Q,U,V] uses the same type.
node field	(nnode,ncx,ncy)	Float64, scaled	Upsampled/interpolated into atmospheric cubes.

Required interfaces

```
abstract type AbstractPopulationModel end
predict_populations!(out, model, atmosphere, geometry, cache)
prepare_line_opacity!(out, line_model, atmosphere, populations_or_nothing, region, cache)
reconstruct_force_balance!(atmosphere, target_log_tau500, boundary_state, eos, opacity, cache)
redistribution_model(line_setup)::AbstractRedistributionModel
synthesize!(spectral_cube, synthesizer, redistribution, atmosphere, populations, line_setup, cache)
apply_observation!(output_cube, observation_model, intrinsic_cube, stokes_set, cache)
forward!(workspace, forward_model, parameters)
residual!(r, objective, parameters, workspace)
checkpoint!(run, solver_state, parameters, diagnostics)
```

Failure contracts

- No NaN/Inf may cross a module boundary. Fail with column, depth, variable, value and parameter-step context.
- EOS or HE3D/MHS non-convergence is a rejected optimizer trial, not silent clipping. Physically safe floors may exist but must be reported.
- Population inference checks channel order, normalization metadata, level count, model hash and input-domain diagnostics.
- Kurucz LTE mode validates line-list format, wavelength medium/unit, species and ion codes, abundance metadata, line-selection margin and file checksum before synthesis.

- Requested PRD or Q/U/V capability is validated before inversion; unsupported combinations fail clearly and never fall back silently.
- All caches are explicit objects. Hidden module-level mutable state is prohibited in threaded code.

5. Numerical strategy

The production parameter vector is too large for a dense STiC-style Jacobian. The plan therefore separates a small-volume reference path from the matrix-free production path.

Parameterization

- Atmospheric variables are represented by vertical nodes whose values are 2D control maps.
- Early cycles use coarse (ncx,ncy) maps and upsample smoothly; later cycles refine the control grid.
- Use bounded transforms or projected steps for physical ranges. Scale parameter groups so solver step sizes are comparable.
- Ptop begins global or very low resolution. Promote it to a spatial map only if synthetic identifiability tests support that choice.

Objective

$$L = L_{\text{data}} + \lambda_z R_z + \lambda_{xy} R_{xy} + \lambda_{\text{prior}} R_{\text{prior}} + \lambda_{\text{boundary}} R_{\text{boundary}}$$

- Ldata uses observation uncertainties, masks and spectral-region weights and is evaluated after the full instrument model.
- Residual packing is Stokes-component-aware from the start. Release 1 activates only I; future Q/U/V use independent masks, uncertainties and weights.
- Rz penalizes excessive first/second variation over log tau500 nodes; Rxy uses quadratic smoothing initially and optional edge-preserving TV later.
- Report every objective component separately. A falling total objective must never hide a worsening data term.

Gradient plan

Stage	Method	Purpose
Reference	Centered finite differences on tiny domains and selected directional derivatives.	Ground truth for interface and gradient checks.
Prototype	Derivative-free or limited-memory optimization on coarse control maps.	Demonstrate closed-loop recovery before committing to adjoints.
Production	Matrix-free gradients: AD where reliable, custom rules/adjoints for force balance and synthesis, and differentiable FFNO bridge or VJP service.	Scale to full parameter fields.
Verification	Taylor remainder tests and dot-product tests for each custom pullback.	Prevent plausible but wrong gradients.

Tiling and distributed execution

Prefer full-volume FFNO execution. When memory requires tiling, specify halo width, overlap window and global-coordinate handling in configuration. Accept a tiling scheme only after population and spectrum errors versus full-volume inference are below declared tolerances in both values and directional derivatives.

6. Phased delivery plan

Each phase ends with an acceptance gate. Later work begins only after its predecessor produces machine-verifiable evidence. Time estimates are intentionally omitted until Phase 0 measures runtime and interface complexity.

Phase 0 - Repository foundation and executable specification

Build: Create the Julia package skeleton, environment lock, configuration schema, canonical types, mock forward components, logging and CI entry point.

Acceptance gate: Package loads; format/lint/unit tests run from one command; a mock end-to-end forward pass produces a deterministic spectral cube; sample configuration validates with actionable errors.

Artifacts: Project.toml; src module tree; test harness; config example; architecture decision records (ADRs); baseline benchmark command.

Phase 1 - Iterative HE3D/MHS reconstruction and Wittmann EOS

Build: Implement both explicit modes. Without B: solve HE3D and skip all magnetic work. With B: compute $\mathbf{J} = \text{curl}(\mathbf{B})/\mu_0$, add $\mathbf{J} \times \mathbf{B}$, and solve MHS. In both modes recalculate ρ with Wittmann, rebuild τ_{500}/z , preserve T and optional B at target tau points, and repeat.

Acceptance gate: HE3D passes with B absent; MHS passes with B present and demonstrably changes force balance through $\mathbf{J} \times \mathbf{B}$; P/ρ converge jointly; force residual and remap errors pass; τ/z stabilizes. Equation, curl discretization and boundary choices are recorded in an ADR.

Artifacts: EOS wrapper; Opacity500; HE3D/MHS solver; Lorentz operator; optional-B remapper; boundary types; mode-specific fixtures and report.

Phase 2 - In-memory FFNO population backend

Build: Expose the trained model as a persistent population service callable from Julia; eliminate hot-loop files and process startup.

Acceptance gate: For recorded atmospheres, Julia-driven results reproduce existing FFNO.Net.py output within numerical tolerance; repeated calls do not reload weights; CPU mock tests and GPU integration tests are separated.

Artifacts: AbstractPopulationModel; PythonCall/RPC backend; metadata validator; record/replay backend; latency/memory benchmark.

Phase 3 - Extensible Julia synthesis kernel

Build: Refactor Forward.jl so population source, atomic/line-list data, line opacity, redistribution policy, formal solution and output are separate. Implement the non-PRD FFNOML intensity path and an LTE Kurucz line-list intensity path, both returning `SpectralCube` with `StokesSet=[]`. Parse and cache line lists outside the hot loop.

Acceptance gate: FFNOML intensity parity passes; Kurucz LTE spectra pass line-by-line and blended-window reference comparisons; LTE regions do not invoke FFNOML; mixed simultaneous NLTE/LTE regions work; PRD and Q/U/V requests fail capability validation; no hot-loop file I/O; mock backends remain substitutable.

Artifacts: LinePhysics and Synthesis modules; AbstractRedistributionModel; AbstractLineOpacityModel; Kurucz parser/index/cache; LTE population/opacity backend; non-PRD FFNOML backend; Stokes-aware SpectralCube; golden fixtures.

Phase 4 - Distributed integrated 3D forward model

Build: Use the single hybrid MPI/Julia-thread production runtime to connect node expansion, distributed force balance/EOS, rank-0-controlled FFNO, synthesis and the component-aware instrument model through reusable workspaces.

Acceptance gate: The same production path runs with one MPI rank or multiple MPI ranks and produces matching scientific outputs; no non-root rank retains the complete CPU atmosphere; repeated evaluations reuse allocations; provenance and rank/thread topology are complete.

Artifacts: ParallelContext; distributed field/tile types; halo exchange; rank-0 GPU service; ForwardModel workspaces; PSF/resampling layer; scaling and allocation report.

Phase 5 - Small-volume inversion prototype

Build: Implement coarse control-map optimization, bounds, objective components, checkpointing and finite-difference directional validation.

Acceptance gate: On an internally generated small 3D cube, the objective falls reproducibly and principal temperature/velocity structure is recovered from multiple initializations; restart reproduces uninterrupted execution.

Artifacts: Prototype solver; multiscale parameter maps; diagnostics bundle; synthetic recovery report.

Phase 6 - Matrix-free gradients and scalable solver

Build: Add distributed module-level VJPs/adjoints, Taylor tests, L-BFGS/trust-region logic and synchronized optimizer decisions on the Phase 4 hybrid runtime.

Acceptance gate: Every custom gradient passes Taylor/dot-product tests; full-domain memory fits target hardware; convergence per forward-call exceeds the prototype; rejected trials recover safely.

Artifacts: Gradient interfaces; scalable solver; distributed scheduler; performance regression suite.

Phase 7 - Independent-physics validation

Build: Generate observations with MULTI3D/RH rather than the inversion surrogate; quantify FFNO and force-balance model mismatch.

Acceptance gate: Bias/error budgets are reported by height, wavelength and spatial scale; acceptance thresholds for scientific use are agreed; retraining contingency is triggered or closed with evidence.

Artifacts: External synthetic benchmark; residual/bias maps; population and spectral comparison; model-card addendum.

Phase 8 - Observational readiness and release

Build: Add robust intensity ingestion, component-aware masks/weights, calibration nuisance parameters, run management and release artifacts. Publish extension guides for PRD and polarized Stokes backends.

Acceptance gate: A blinded observation-like dataset runs from configuration to report; failure recovery and provenance are tested; user and developer documentation permit a fresh checkout run.

Artifacts: Versioned release; operator guide; API docs; example run; validation dossier.

7. Test and validation strategy

Layer	Required tests	Evidence retained
Types / nodes	Shape, orientation, monotonic grid, interpolation exactness at nodes, bounds, serialization round trip.	Unit-test output and small golden fixtures.
EOS / opacity	Known thermodynamic points, unit conversions, positivity, continuity and edge-domain behavior.	Reference tables with provenance.
HE3D / MHS	No-B HE3D path; B-present Lorentz/MHS path; force residuals, P/rho contraction, tau-z reconstruction, optional-B preservation and manufactured solutions.	Mode-specific comparisons, force/remap plots and numerical summary.
FFNO bridge	Parity with Python, model metadata, repeatability, batching, halos, failure on bad channels.	Recorded request/response fixtures and benchmark.
Synthesis	FFNOML intensity parity; Kurucz parsing and LTE line-opacity references; isolated lines and blends; mixed NLTE/LTE regions; Stokes-axis contract; capability failures; thread safety.	Golden spectra, selected-line inventories, line-list checksums, substitution tests and numerical differences.
Future seams	Mock PRD state and mock IQUV solver traverse ForwardModel, observation operator and residual packing without core edits.	Architectural substitutability test; no scientific-validity claim.
Observation model	PSF normalization, delta-input response, resampling conservation, masks and weights.	Analytic and golden tests.
Integrated forward	Determinism, allocation stability, input mutation checks, full vs tiled parity.	End-to-end fixture and profile.
Gradients	Centered directional finite difference, Taylor order, adjoint dot-product tests.	Per-module gradient report.
Inversion	Exact-model recovery, noise ladder, initial-condition sensitivity, checkpoint/restart.	Recovery metrics and objective histories.
Independent physics	MULTI3D/RH-generated spectra, out-of-distribution diagnostics, surrogate bias.	Scientific validation dossier.

Minimum synthetic ladder

- Uniform atmosphere: catches axes, units and spectral normalization errors.
- Single smooth horizontal mode: tests true spatial coupling and halo behavior.
- Corrugated tau surfaces with known velocities: tests HE3D/MHS geometry and Doppler shifts.
- Realistic simulation subvolume: tests domain range, memory and optimizer behavior.
- Independent MULTI3D/RH spectra: measures surrogate/model mismatch instead of inverse crime performance.

Release tolerances

Numerical tolerances must live in configuration or test constants with a physical justification. Avoid a single universal relative tolerance: populations spanning many orders of magnitude require log-space, absolute-floor and line-impact metrics.

8. Risks and decision gates

Risk	Early signal	Mitigation / decision
z-scale convention mismatch	FFNO populations change strongly when reconstructed z uses a plausible alternate zero point.	Reproduce training convention; test invariant/relative coordinate features; retrain on HSE-reconstructed inputs if necessary.
Force balance cannot represent atmosphere	Large reconstruction errors in rho/ne/z despite correct EOS.	Quantify spectral effect; add pressure correction or broaden physics only with identifiability evidence.
Force-equation incompatibility	HE3D horizontal gradients or MHS Lorentz force are not compatible with a scalar P/boundaries.	Use curl/integrability and manufactured tests; reject inconsistent states; document the chosen equation and boundaries.
Dense cross-language hot loop	Serialization/startup dominates runtime or allocations grow.	Persistent worker, shared memory/DLPack where safe, larger batches; migrate inference representation only after parity tests.
FFNO halo artifacts	Tile seams or changing answers with halo width.	Full-volume path, larger halos, overlap windows, distributed spectral operations.
Wrong gradients	Objective sometimes rises despite line search; derivative tests fail by scale/domain.	Keep finite-difference oracle; module-level Taylor tests; do not enable production gradient until passing.
PRD/Stokes leakage	Core types assume scalar spectra or redistribution logic enters the optimizer.	Enforce SpectralCube/StokesSet and LinePhysics interfaces; keep mock extension tests in CI.
Kurucz convention mismatch	LTE spectra differ by wavelength offset, strength or wings despite matching atmosphere.	Record air/vacuum wavelength convention, units, abundance set, species mapping, log(gf), damping/broadening interpretation and compare isolated-line golden cases before blends.
Parameter non-identifiability	Different atmospheres fit spectra equally well; Ptop drifts.	Coarse-to-fine maps, priors, restrict variables, multi-line observations and ensemble experiments.
Surrogate exploitation	Optimizer finds FFNO out-of-domain atmospheres with artificially low residuals.	Trust-region/domain penalties, input-distribution diagnostics, independent-physics validation.
Memory/runtime ceiling	One forward/gradient pass exceeds target GPU memory or wall time.	Measure Phase 0 baseline; explicit workspaces, mixed precision only after accuracy tests, distributed domain decomposition.

Mandatory go/no-go gates

- After Phase 1: HE3D must work with B genuinely absent; MHS must work with supplied B and Lorentz force; P/rho and force residuals must converge; T and optional B must be preserved on target tau points.
- After Phase 4: full and deployable tiled forward modes must agree to a scientifically acceptable tolerance.
- After Phase 5: exact-model synthetic recovery must work before gradient optimization begins.
- After Phase 7: independent-physics bias determines whether the system is ready for observations or requires FFNO retraining/model changes.

9. Repository and workflow conventions

Suggested repository layout

```
FFNOInversion.jl/ Project.toml # pinned Julia environment src/ FFNOInversion.jl # public API
only Types.jl Nodes.jl ForceBalance.jl EOS.jl Opacity500.jl PopulationModels.jl LinePhysics.jl
Synthesis.jl Observations.jl ForwardModel.jl Objectives.jl Solvers.jl IO.jl Diagnostics.jl test/
unit/ integration/ gradients/ regression/ configs/ # versioned examples, never machine-local
paths scripts/ # thin CLIs; no core physics benchmarks/ docs/adr/ # architecture decision records
artifacts/ # small recorded fixtures; large data externally located
```

Definition of done for every change

- Public types/functions have docstrings, shapes and units; new configuration keys are documented and validated.
- No public API hard-codes one Stokes component or one redistribution treatment; capability declarations and mock substitution tests remain passing.
- Tests cover nominal and failure behavior; reference data include provenance and generation command.
- No unnecessary file I/O, model reload or large allocation is introduced into forward!/residual!.
- Benchmarks are compared when a hot path changes; scientific differences are summarized numerically and visually.
- Checkpoint compatibility is preserved or a migration is supplied; completed phases update the ADR and plan status.

Configuration principles

- One declarative TOML/YAML run configuration references separate atomic, observation, model and solver sections.
- Every spectral region selects its population mode. LTE regions name Kurucz line-list files, wavelength convention, wing-selection margin, abundance set and broadening policy; FFNOML regions name their model/transition metadata.
- Paths are resolved relative to the config or through explicit environment profiles; source code contains no personal absolute paths.
- A dry-run command prints resolved shapes, units, memory estimate, checkpoint/model hashes and enabled physics without running inversion.

10. Human-Codex execution protocol

This section makes the document directly actionable by later Codex sessions while preserving human control of scientific choices.

At the start of each phase

- Read this plan, current README, relevant source modules and repository instructions. Treat documents as technical context; the current human request controls scope.
- Inspect git status and preserve unrelated user changes. Create a short phase plan tied to the acceptance gate.
- Record open scientific choices in an ADR. Ask the human only when a choice materially changes physics, data meaning or external interfaces.
- Implement the smallest vertical slice that produces evidence, then expand; do not begin later-phase infrastructure speculatively.

Required phase handoff

Item	Handoff content
Outcome	What now works, stated in scientific/user terms.
Changed files	Clickable paths and the role of each material change.
Verification	Exact commands, passed tests, numerical comparisons and rendered diagnostics.
Known limitations	Unvalidated regimes, performance limits, approximation boundaries and deferred work.
Acceptance gate	Pass/fail assessment against this document; no subjective 'looks good'.
Next phase	The first safe action and any required human decision.

Implementation rules for agentic work

- Never infer axis order or units from array size; assert metadata at boundaries.
- Never replace a scientific reference with a mock in production code; mocks exist only behind explicit test backends.
- Never optimize away validation fixtures. Parity tests precede refactors of EOS, FFNO bridge or synthesis.
- Never claim inversion success from an inverse-crime test alone. Independent-physics validation remains mandatory.
- Keep commands reproducible and non-interactive; generated results record source revision and configuration checksum.

11. Phase 0 kickoff backlog

The following is the concrete starting backlog for the next work session.

ID	Task	Acceptance check
P0-01	Create FFNOInversion.jl package skeleton with module boundaries from Section 2.	Julia --project -e 'using FFNOInversion' succeeds.
P0-02	Define Grid3D, Atmosphere3D with optional B, ForceBalanceMode, HE3DBoundaryState(rho0,P0), ObservationCube, NodeField, ModelParameters and workspace.	Constructors select HE3D when B is absent and MHS when B is present; no dummy magnetic arrays.
P0-02b	Define SpectralCube, StokesSet, CapabilityManifest, AbstractRedistributionModel and AbstractSynthesizer with non-PRD/intensity defaults.	Core shapes include a Stokes axis; mock PRD/IQUV backends plug in without changing ForwardModel types.
P0-03	Define AbstractPopulationModel and AbstractSynthesizer plus deterministic mock implementations.	Mock full forward produces a fixed cube and is allocation-stable after warm-up.
P0-04	Implement node expansion on log tau500 and coarse horizontal control maps.	Analytic constant/linear fields are reproduced; bounds and gradients are tested.
P0-05	Add configuration schema and dry-run CLI.	Example config resolves without personal absolute paths and prints shapes/memory estimate.
P0-06	Set up unit/integration test entry points, formatting and a baseline benchmark.	One documented command runs the local quality gate.
P0-07	Write ADR-001: coordinates/units; ADR-002: Python bridge; ADR-003: HE3D/MHS; ADR-004: PRD/Stokes extension boundaries and capability manifest.	Human reviews scientific conventions before Phase 1 physics code.
P0-08	Inventory current Forward.jl and FFNONet.py callable seams without behavior changes.	Short interface report lists reusable kernels, global state, I/O and refactor hazards.

Phase 0 completion evidence

A single command constructs a small 3D node field, expands it to an atmosphere, passes it through mock population and synthesis backends, applies a mock observation model, computes residual components, writes a restartable checkpoint, and reproduces the identical result after reload.

First human decisions

- Confirm target lines and viewing geometry. For each region choose FFNOML non-LTE selected-line synthesis or LTE Kurucz line-list synthesis. Release 1 is intensity-only and non-PRD; list likely future PRD lines and Stokes instruments only to validate interfaces.
- Confirm the source and convention of the 500 nm continuum opacity reference implementation.
- Confirm whether $\text{grad}(P) = -\rho \cdot g$ is intended literally in all three spatial components or as the vertical hydrostatic equation applied across a 3D volume.
- Confirm whether rho0 and P0 are scalars or boundary maps, and on which boundary they are specified.
- Confirm the geometrical-height zero convention used to train FFNO and the required initial domain size/hardware target.
- Choose whether FFNOInversion.jl lives inside this repository or as a sibling Julia package with this repository as a model dependency.

Plan status: Phases 0-3 and the hybrid runtime foundation are implemented. Baseline v1.4 defines one production execution architecture: MPI plus Julia threads, with GPU work launched only by global rank 0. A one-rank run is a test/development topology of this same path, not a separate serial implementation.

12. Hybrid MPI, threading and GPU execution architecture

There is exactly **one production execution architecture**. MPI distributes spatial ownership between processes and nodes; Julia threads perform independent column work inside each rank's local tile; global MPI rank 0 alone owns the GPU-service control plane and is the only rank that launches GPU processes. A one-rank run exercises this same distributed code path with one MPI tile; it is not a second serial forward implementation.

Execution hierarchy

```
scheduler allocation -> MPI ranks -> one spatial tile per rank -> Julia threads over local columns; global rank 0 -> persistent FFNO GPU service or nested multi-GPU job
```

Architecture invariant

Application-level forward, objective and inversion routines must accept rank-owned distributed state. Existing Phase 0-3 routines remain low-level scientific kernels called inside the hybrid path; they must not be assembled into a competing monolithic application route. MPI rank count and threads per rank change deployment topology only, never the scientific control flow.

Layer	Primary responsibility	Ownership rule
MPI	Partition the x-y field of view, exchange halos, reduce objectives/gradients and coordinate iteration state.	A rank owns only its local tile, halos and workspaces; it does not retain the complete CPU atmosphere.
Julia threads	Run independent column synthesis, Kurucz LTE work, column-local EOS work, vertical regularization and local spectral operations.	Threads share immutable rank-local caches; every thread has private mutable scratch storage.
GPU service	Evaluate FFNO inference and later its vector-Jacobian product (VJP).	Only global MPI rank 0 launches and controls GPU processes. The service is persistent across inversion iterations.
I/O	Read the two inputs and write the synthesis and atmosphere outputs.	Rank 0 controls metadata and partitioning; data are streamed/scattered by tile or read as assigned hyperslabs.

Rank and thread topology

The number of MPI ranks is not required to equal the number of pixels or CPU cores. A rank owns a multi-column tile, and its Julia threads process columns within that tile. On a 256-core allocation, **16 MPI ranks x 16 Julia threads** is a valid and likely useful starting point. Fewer ranks reduce halo surfaces, message count, duplicated atomic/line-list caches and MPI runtime overhead.

Example allocation	Candidate layout	Reason to benchmark
1 node, 64 cores, 2 NUMA domains	2 ranks x 32 threads, or 4 ranks x 16 threads	One rank per NUMA domain minimizes remote-memory traffic; smaller thread teams can improve bandwidth scaling.
4 nodes, 64 cores/node	4 ranks/node x 16 threads = 16 ranks x 16 threads	Balances tile size, cache duplication, halo traffic and thread-level column concurrency.
256 cores, memory-heavy atomic models	8 ranks x 32 threads	Fewer copies of read-only atomic and Kurucz data may reduce memory pressure.
256 cores, weak thread scaling	32 ranks x 8 threads	More independent MPI tiles may outperform large thread teams.

The default launcher should be NUMA-aware: bind each MPI rank to a socket or NUMA domain and bind its Julia threads to distinct cores. The project will benchmark at least two rank/thread layouts on the target machine; topology is configuration and launch policy, not scientific code.

Threading contract for Forward.jl

Forward.jl may use **Threads.@threads** over the rank-local columns. Thread safety is an explicit acceptance requirement: opacity arrays, formal-solver buffers, interpolation state, Kurucz scratch data and accumulation buffers are private to a thread. Parsed line lists, atomic constants and immutable lookup tables may be shared.

Avoid nested oversubscription. When Julia threads own column concurrency, set OpenMP and BLAS libraries to one thread per MPI rank process. A kernel may deliberately use threaded BLAS instead, but it must disable outer Julia concurrency for that region; both layers must never independently claim all cores.

12.1 Distributed data ownership and communication

A two-dimensional Cartesian MPI decomposition is the target abstraction. Each rank receives a rectangular x-y tile spanning the full vertical and synthesis wavelength dimensions required for its work. The implementation may initially adapt the proven x-slab utilities from the **charge** branch, but public distributed-field interfaces must not assume slabs.

Operation	Communication pattern	Local computation
HE3D/MHS	Neighbour halo exchange plus distributed reductions/linear-solver communication.	Force, EOS and opacity work on tile interiors and boundaries.
FFNO population inference	Tiles are staged to the rank-0-controlled GPU service; population tiles are returned to their owning ranks.	CPU ranks retain only local atmospheres and returned local populations.
NLTE and Kurucz synthesis	No communication for independent rays/columns after populations and atmosphere are local.	Threaded column loop inside every MPI rank.
Spatial PSF	Exchange halo widths determined by the PSF support.	Convolve the owned output region; do not interpolate or change the forward grid.
Spectral PSF and weights	Normally none between spatial tiles.	Convolve wavelengths, then apply wavelength/Stokes and spatial weights for chi-squared.
Horizontal regularization	Exchange parameter halos.	Evaluate local penalty and local gradient.
Objective/gradient	MPI.Allreduce for scalar objective, norms and global parameters.	Residuals and atmospheric gradients remain tile-local.
Final outputs	Collective hyperslab writes are preferred; root gathering is a small-run fallback.	Each rank supplies only its owned synthesis and atmosphere tile.

No process loads the entire CPU model

Rank 0 reads configuration and global metadata, chooses decomposition, and streams/scatters data in bounded tiles. For production volumes, parallel HDF5 hyperslab I/O is preferred: rank 0 assigns ranges while every rank reads or writes only its range. This preserves rank-0 control without making rank 0 a memory or bandwidth bottleneck.

FFNO3D is spatially coupled, so the full spatial context must exist somewhere during inference. In the initial single-GPU backend the complete FFNO tensor may exist on the rank-0-controlled GPU, while no CPU rank permanently owns the full model. If it does not fit on one GPU, rank 0 launches a nested distributed-GPU backend using the distributed FFT/transposition approach demonstrated in **charge**. Both modes implement the same population-service interface.

Synchronization protocol

The **charge** branch's control-channel pattern is adopted. During a nested GPU/Slurm/NCCL launch, non-root ranks must not wait with outstanding MPI requests on systems where this can deadlock. Rank 0 broadcasts a control endpoint; non-root ranks wait on the socket; rank 0 runs the GPU stage and signals completion; all ranks then re-enter MPI at the same named synchronization point.

Every forward/gradient evaluation follows one deterministic state machine:

```
tile update -> HE3D/MHS + EOS -> rank-0 FFNO -> population scatter -> threaded synthesis ->
PSF/residual -> regularization -> MPI Allreduce -> optimizer decision
```

12.2 Configuration, testing and acceptance

The scheduler selects MPI process count; the configuration records intended thread behavior and validates the observed runtime topology. Development and parity tests may use one MPI rank, but they invoke the same distributed application entry point used by multi-rank production runs.

```
[parallel] decomposition = "cartesian_2d" threads_per_rank = 16 threaded_synthesis = true
threaded_eos = true threaded_spectral_psf = true gpu_launcher_rank = 0 gpu_backend = "root_gpu"
# future: distributed_gpu parallel_hdf5 = true # Example launch for 256 allocated cores: #
JULIA_NUM_THREADS=16 OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 \ # mpiexec -n 16 julia
--threads=16 inversion.jl config.toml
```

Required hybrid-runtime tests

- Scientific topology parity: one-rank and multi-rank executions of the same hybrid application path agree within declared numerical tolerances.
- Topology parity: changing ranks x threads while keeping total cores comparable does not change the scientific result.
- Memory audit: after input distribution, no CPU rank retains a complete atmosphere, population or synthesis cube.
- Thread-safety stress test: repeated synthesis under multiple thread schedules is deterministic and race-free.
- Halo parity: distributed HE3D/MHS, spatial PSF and horizontal regularization match a small-domain monolithic reference.
- GPU protocol test: rank 0 alone launches the persistent GPU service; injected GPU failure is reported coherently by every rank without deadlock.
- Restart parity: a checkpoint written under one safe rank/thread topology can restart under another compatible topology.
- Scaling study: compare at least two layouts, such as 16 x 16 and 32 x 8 on 256 cores, reporting synthesis throughput, communication fraction, memory per rank and complete forward time.

Phase 4 acceptance amendment

Phase 4 is accepted only when the distributed containers, 2D ownership model, thread-local Forward.jl workspaces, halo operations, rank-0-only GPU control service and synchronized failure handling are integrated into the single production path. A full synthetic cube must agree between one-rank and multi-rank topologies of that path, and the scaling report must recommend a measured rank/thread topology for the first target machine.